

AGONETEST: Automated creation and assessment of Unit tests leveraging Large Language Models

Andrea Lops

Polytechnic University of Bari - Wideverse
Italy
andrea.lops@poliba.it

Azzurra Ragone

University of Bari
Italy
azzurra.ragone@uniba.it

Fedelucio Narducci

Polytechnic University of Bari
Italy
fedelucio.narducci@poliba.it

Michelantonio Trizio

Wideverse
Italy
michelantonio.trizio@wideverse.com

ABSTRACT

Software correctness is crucial, with unit testing playing an indispensable role in the software development lifecycle. However, creating unit tests is time-consuming and costly, underlining the need for automation. Leveraging Large Language Models (LLMs) for unit test generation is a promising solution, but existing studies focus on simple, small-scale scenarios, leaving a gap in understanding LLMs' performance in real-world applications, particularly regarding integration and assessment efficacy at scale. Here, we present AGONETEST, a system focused on automatically generating and evaluating complex class-level test suites. Our contributions include a scalable automated system, a newly developed dataset for rigorous evaluation, and a detailed methodology for test quality assessment.

CCS CONCEPTS

• **Software and its engineering** → **Automatic programming;**
Software testing and debugging.

KEYWORDS

Software Testing, Large Language Model, Automatic Assessment

ACM Reference Format:

Andrea Lops, Fedelucio Narducci, Azzurra Ragone, and Michelantonio Trizio. 2024. AGONETEST: Automated creation and assessment of Unit tests leveraging Large Language Models. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, October 27-November 1, 2024, Sacramento, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3691620.3695318>

1 INTRODUCTION

Unit testing is a fundamental stage within the software testing lifecycle, essential for validating individual units of source code to ensure their correctness and reliability. However, creating unit tests

is a labor-intensive and time-consuming task requiring significant effort from developers and test engineers.

The advent of Large Language Models (LLMs) has brought substantial advances in software development, including automating unit test generation [2]. Although LLMs have shown potential to improve efficiency, existing studies often focus on generating unit tests for isolated methods, providing an incomplete picture of LLM performance in more complex real-world scenarios [4]. Moreover, manually integrating and executing these generated unit tests within original projects limits scalability and overall efficiency [3]. This manual process hinders the application of automated testing in larger and more complex codebases typical of real-world projects.

To bridge these gaps, we introduce AGONETEST, an automated system for generating and evaluating test suites that reflect real-world complexities. Our primary contribution is a fully automated system that covers the entire process from test generation to assessment. Furthermore, we offer a new dataset, CLASSES2TEST¹, based on METHODS2TEST [6], allowing a rigorous comparison between human-written class-level tests and those generated by LLM. The system evaluates the generated test suites using advanced metrics such as code coverage, mutation coverage [1], and detection of test smells [5].

2 PROBLEM ADDRESSED

The primary challenges we address with AGONETEST are threefold:

Limited Scope of Prior Studies: Existing studies primarily focus on small-scale or simplistic unit test generation tasks. These studies do not provide a comprehensive evaluation of LLMs for complex, class-level code structures.

Lack of Automation in Test Generation: Automating the generation of test suites for sophisticated, class-level code structures in Java projects to better reflect real-world software development scenarios.

Comprehensive Evaluation: Providing a comprehensive methodology for evaluating the quality and effectiveness of LLM-generated test suites, moving beyond manual integration and execution.

To address these challenges, this study poses the following research question (RQ):

RQ: To what extent is it possible to implement an automated end-to-end process for generating test suites?

¹<https://github.com/lopsandrea/classes2test>



This work is licensed under a Creative Commons Attribution-ShareAlike International 4.0 License.
ASE '24, October 27-November 01, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1248-7/24/10
<https://doi.org/10.1145/3691620.3695318>

Table 1: Compilation and passing rates of classes per experiment

model	prompt name	Build	Pass	Total Rejected
gpt-3.5-turbo	zeroshot	64 (68.08%)	36 (38.29%)	58 (61.70%)
gpt-3.5-turbo	oneshot	65 (69.14%)	35 (37.23%)	59 (62.76%)
gpt-4	zeroshot	76 (80.85%)	29 (30.85%)	65 (69.14%)
gpt-4	oneshot	76 (80.85%)	28 (29.78%)	66 (70.21%)
human	-	94 (100%)	94 (100%)	0 (0%)

3 APPROACH

We developed AGONETEST², an automated system to generate and evaluate unit test suites for Java projects, supported by a structured methodology to assess their quality. Our approach builds upon the METHODS2TEST dataset as a foundation, extending it to create CLASSES2TEST, designed to compare human-written class-level tests with those generated by LLMs. The system encompasses several key phases:

Sample Projects Selection: We used an annotated dataset of Java projects from GitHub, the selected repositories are representative of real-world projects.

Configuration Parameters Elicitation: The system extracts essential parameters, such as Java version, testing framework, and dependencies, from build configuration files. Parsing these files allows us to gather crucial configuration details for generating context-aware prompts for the LLMs.

Prompt Creation: User-supplied prompt templates guide the LLM in generating unit tests. Prompts are populated by replacing variables within the prompt skeletons with specific contextual information from the project. For instance, placeholders such as `{testing_framework}`, `{java_version}`, and `{focal_class}` are dynamically replaced with the extracted parameters to generate accurate and contextually relevant prompts.

Test Suite Generation: AGONETEST can call multiple LLMs through a standardized API, using created prompts to generate test classes. The system extracts the relevant test class code from the LLM's response, ensuring only the necessary code is retained for integration.

Test Suite Assessment: Generated test suites are evaluated using metrics such as instruction, branch, line, method, and mutation coverage, calculated using libraries such as JaCoCo and PiTest. Test smells are detected using tools like tsDetect. The results are compiled into a comprehensive report, providing detailed information on the quality and effectiveness of the generated test suites across various prompts and LLMs.

4 MAIN RESULTS

To answer our research question, we conducted experiments using AGONETEST on a sample of 10 Java repositories from the CLASSES2TEST dataset. We evaluated the performance of different LLMs and prompt strategies in generating and assessing unit test suites for these repositories.

RQ: To what extent is it possible to implement an automated end-to-end process for generating test suites? Our experiments demonstrate that AGONETEST can handle the end-to-end process autonomously. However, certain challenges remain. Table 1 highlights the compilation success rates and passing test percentages for the generated test suites. Approximately 75% of the generated test classes compiled successfully, while only 34% of generated tests passed. Detailed contextual information from project configuration files ensures high levels of automation and reduces manual intervention.

Despite these achievements, there are still areas for improvement, particularly in enhancing the robustness of the generated tests to increase the percentage of passing tests. Test failures were often due to incorrect imports or syntax errors, rather than discovering actual bugs.

Preliminary experiments indicate that the generated test suites perform comparably to human-written tests in terms of code and mutation coverage. However, this aspect requires further study for conclusive results.

5 CONCLUSION AND FUTURE WORK

AGONETEST presents a novel framework for the automated generation and rigorous evaluation of unit test suites using LLMs, addressing gaps in current methodologies and scaling to real-life software complexities. The results illustrate the framework's capability to generate viable test suites and provide comprehensive quality assessments. However, further automation for error correction and improved robustness of generated tests are necessary. Future work will refine context-aware validation and systematically research optimal LLM-prompt combinations. The ultimate goal is to equip software testers and developers with an automated, scalable tool capable of generating robust, high-quality unit test suites, minimizing manual effort, and maximizing testing efficiency.

ACKNOWLEDGMENTS

This work is partially funded by the PRIN TRex-SE: Trustworthy Recommenders for Software Engineers under grant no. 2022LKJWHC.

REFERENCES

- [1] Mauricio Aniche. 2022. *Effective Software Testing: A developer's guide*. Simon and Schuster, New York, NY, USA.
- [2] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. arXiv:2212.14834 [cs.SE]
- [3] Vitor Guilherme and Auri Vincenzi. 2023. An initial investigation of ChatGPT unit test generation capability. In *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing* (Campo Grande, MS, Brazil) (SAST '23). Association for Computing Machinery, New York, NY, USA, 15–24. <https://doi.org/10.1145/3624032.3624035>
- [4] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning. *2023 IEEE/ACM 45th International Conference on Software Engineering* (2023).
- [5] Fabio Palomba, Dario Di Nucci, Annibale Panichella, Rocco Oliveto, and Andrea De Lucia. 2016. On the diffusion of test smells in automatically generated test code: an empirical study. In *Proceedings of the 9th International Workshop on Search-Based Software Testing*. <https://doi.org/10.1145/2897010.2897016>
- [6] Michele Tufano, Shao Kun Deng, Neel Sundaresan, and Alexey Svyatkovskiy. 2022. Methods2Test: a dataset of focal methods mapped to test cases. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*. Association for Computing Machinery, New York, NY, USA, 299–303. <https://doi.org/10.1145/3524842.3528009>

²<https://arxiv.org/abs/2408.07846>